Introduction

Count-Min Sketch (CMS) is applied to flag IP addresses whose estimated frequency meets or exceeds a given threshold, as a first-layer filtering mechanism for detecting unusually active addresses. Because CMS uses hash functions to map multiple IP addresses into shared counters, it can overestimate frequencies, which may cause some IP addresses to be incorrectly flagged, causing false positives. This experiment investigates how the accuracy of a CMS depends on its memory usage by varying two parameters: the number of hash tables and the size of each table. Accuracy is defined as precision — the proportion of flagged IP addresses that genuinely exceed the threshold — comparing the set of IPs flagged by CMS against those that actually crossed the threshold frequency. This measures whether CMS correctly identifies the IP addresses generating an unusually large number of requests, rather than counting individual flagging events, since the same IP may be flagged multiple times as the stream is processed. Since CMS never underestimates frequencies, it produces no false negatives as errors only occurred through false positives caused by hash collisions.

Methodology

Data Generation

The program generateIPs2.c outputs repeated IP addresses. Each of 500 unique IP addresses is generated randomly, and each one is repeated between one and six times. Since CMS estimates cumulative frequencies, the final frequency estimates for each IP are independent of stream order, although the exact point during processing at which an IP first becomes flagged may vary. Thus, the ordering of IPs generated does not matter. The total number of IP addresses is computed before the file is opened, so the file header accurately reflects the true number of IPs. This process is repeated to generate three independent input files (cmsinput1.txt to cmsinput3.txt), each seeded differently by calling srand(time(NULL)) once before all three files are generated, ensuring distinct datasets for averaging.

Experimental Design

cmsExperiment.c is run once per input file, with output redirected to results1.txt through results3.txt. For each run, the program sweeps over all combinations of num_tbls from 1 to 20 (incrementing by 1) and size_tbls from 10 to 1000 (incrementing by 50). For each combination, a new CMS structure is allocated and all IP addresses are inserted.

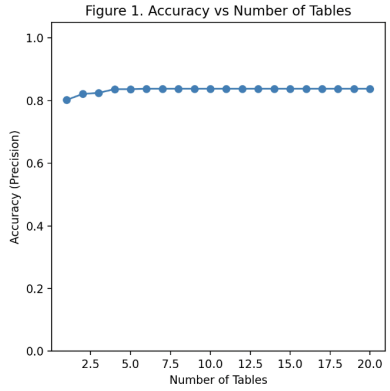
Accuracy

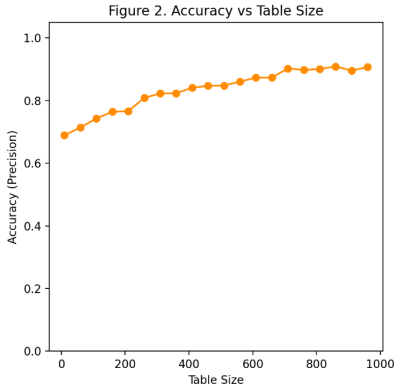
After all addresses are inserted, each flagged entry in the stream is examined. To avoid counting the same IP address multiple times, an IP is only evaluated if it has not already been seen among earlier stream positions with the same flag state (0 for unflagged and 1 for flagged). For each unique flagged IP, calc_freq is used to compute its true frequency across the entire input. If the true frequency meets or exceeds the threshold, it is counted as a true positive; otherwise it is a false positive. Accuracy is defined as precision: True Positives / (True Positives + False Positives). If no IPs are flagged, precision is recorded as 1.0 since there are no false positives. Memory usage for each configuration is computed as num_tbls × size_tbls × sizeof(int) bytes.

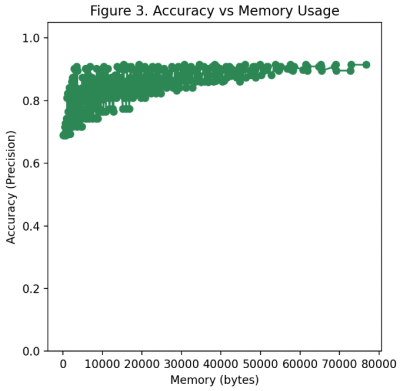
Plotting

The three results files are averaged across matching configurations using Python. Three line graphs are produced: accuracy versus number of tables (averaged over all table sizes), accuracy versus table size (averaged over all numbers of tables), and accuracy versus total memory usage.

Results







Discussion

As table size increases, accuracy rises from approximately 0.7 to 0.9, plateauing near a table size of 1000 (Figure 2). This is explained by hash collisions. Each CMS table maps all IP addresses into size_tbls buckets, so when the table is small, many distinct IPs share the same counter. When IPs collide in a table, incrementing any IPs in that bucket increments the shared counter, causing CMS to overestimate their frequencies. This overestimation produces false positives — IPs that appear to exceed the threshold when they do not. As table size grows, the probability of two IPs mapping to the same bucket decreases, so counters more accurately reflect individual IP frequencies, reducing false positives and increasing precision. The plateau occurs because beyond a certain table size, collisions become rare enough that further increases in size yield diminishing increases in accuracy.

However, increasing the number of tables has little effect on accuracy, remaining flat around 0.82 (Figure 1). In theory, increasing the number of tables decreases the probability of large overestimation errors, since a false positive requires collisions to occur across multiple tables simultaneously, which will be increasingly unlikely as more hash tables with independent hash functions are used. However, in this implementation all tables use the same linear hash formula (ip*(rands[i]%MAX_RAND))%size, where the random multipliers are drawn once from the same seeded sequence and reused across all configurations. As the hash functions are not fully independent, collision patterns across tables may remain correlated, reducing the effectiveness of the minimum operation. As a result, adding more tables does not significantly decrease false positives.

Figure 3 shows that accuracy is dominated by table size rather than number of tables, consistent with Figures 1 and 2. At low memory values the spread of accuracy scores is wide — the same memory space can be achieved by many different combinations of few large tables or many small tables, and small tables produce high collision rates and therefore more false positives regardless of how many tables there are. As memory increases the spread narrows and the lower bound on accuracy rises, because higher memory values require larger table sizes which reduce collisions. This has direct implications for CMS as a first-layer filtering mechanism: the role of CMS is not to make final decisions but to cheaply narrow down a large stream to a set of suspicious IPs for more expensive downstream verification. False positives are acceptable provided they are not excessive, since IPs incorrectly passed through will be cleared at the next layer. However, the results suggest that memory should be invested primarily in larger table sizes rather than more tables, as this delivers the greatest reduction in false positives per byte of memory allocated, making the first-layer filter more precise.